파이썬 함수

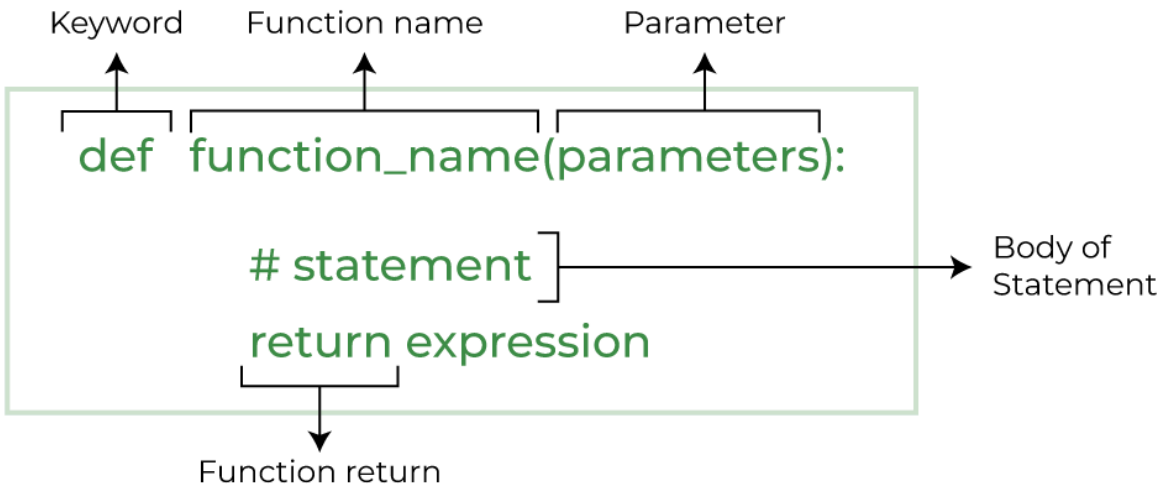
파이썬 함수는 특정 작업을 수행하는 명령문 블록입니다. 자주 수행되거나 반복적으로 수행되는 작업을 모아 함수를 만드는 것이 핵심입니다. 이렇게 하면 서로 다른 입력값에 대해 동일한 코드를 반복해서 작성하는 대신, 함수 호출을 통해 함수에 포함된 코드를 반복해서 재사용할 수 있습니다.

함수 사용의 이점

- 코드 재사용
- 코드 길이 감소
- 코드의 가독성 향상

파이썬 함수 선언

함수를 선언하는 구문은 다음과 같습니다.



매개변수

매개변수는 함수를 선언할 때 괄호 안에 정의되는 변수입니다.

인수

인수는 함수가 호출될 때 전달되는 값입니다. 함수나 메서드에 입력으로 전달되는 변수, 값 또는 객체일 수 있습니다.

설명:

- **def:** 함수 정의를 시작합니다.
- **function_name:** 함수의 이름.
- **매개변수:** 함수에 전달되는 입력
- **:** : 함수 본문의 시작을 나타냅니다.
- **들여쓰기된 코드:** 호출 시 실행되는 함수 본문입니다.

예: 간단한 예를 통해 이를 이해해 보겠습니다. 여기서는 def를 사용하여 호출 시 환영 메시지를 출력하는 함수를 정의합니다.

```
def fun():
    print("Welcome to GFG")
```

파이썬에서 함수 호출하기

Python에서 함수를 생성한 후에는 Python 함수 이름 뒤에 해당 함수의 매개변수를 담은 괄호를 붙여 호출할 수 있습니다. 아래는 def 함수를 Python으로 호출하는 예시입니다.

```
def fun():
    print("Welcome to GFG")

# Driver code to call a function
fun()
```

파이썬 함수 인수 Function Arguments

인수는 함수의 괄호 안에 전달되는 값입니다. 함수는 쉼표로 구분된 여러 개의 인수를 가질 수 있습니다.

인수가 있는 함수의 구문:

```
def function_name(parameter: data_type) -> return_type:
  """Docstring"""
  # body of the function
  return expression
```

함수 선언에서 data_type 과 return_type 은 선택 사항이므로 동일한 함수를 다음과 같이 작성할 수도 있습니다.

```
def function_name(parameter) :
  """Docstring"""
  # body of the function
  return expression
```

예를 들어 설명해 보겠습니다. Python에서 함수에 인수로 전달된 숫자가 짝수인지 홀수인지 확인하는 간단한 함수를 만들어 보겠습니다.

```
def evenOdd(x: int) ->str:
    if (x % 2 == 0):
        return "Even"
    else:
        return "Odd"

print(evenOdd(16))
print(evenOdd(7))
```

산출

위 함수는 다음과 같이 **type_hints** 없이 선언될 수도 있습니다 .

```
def evenOdd(x):
    if (x % 2 == 0):
        return "Even"
    else:
        return "Odd"

print(evenOdd(16))
print(evenOdd(7))
```

산출

파이썬 함수 인수의 유형

Python은 함수 호출 시 전달될 수 있는 다양한 유형의 인수를 지원합니다. Python에는 다음과 같은 함수 인수 유형이 있습니다.

- **기본 인수**
- **키워드 인수(명명된 인수)**
- **위치적 인수**
- **임의 인수** (가변 길이 인수 *args 및 **kwargs)

각 유형에 대해 자세히 살펴보겠습니다.

기본 인수

기본 인수는 함수 호출 시 해당 인수에 대한 값이 제공되지 않을 경우 기본값을 사용하는 매개변수입니다. 다음 예제는 Python에서 함수를 작성할 때 기본 인수를 사용하는 방법을 보여줍니다.

```
def myFun(x, y=50):
    print("x: ", x)
    print("y: ", y)


myFun(10)
```

산출

C++ 기본 인수처럼 함수의 인수는 개수에 제한 없이 기본값을 가질 수 있습니다. 하지만 기본 인수가 있으면 오른쪽에 있는 모든 인수도 기본값을 가져야 합니다.

키워드 인수

이 아이디어는 호출자가 매개변수의 순서를 기억할 필요가 없도록 인수 이름과 값을 지정할 수 있도록 하는 것입니다.

```
def student(fname, lname):
    print(fname, lname)

student(fname='Geeks', lname='Practice')
student(lname='Practice', fname='Geeks')
```

산출

임의의 키워드 인수

Python에서 임의의 키워드 인수, *args 및 **kwargs는 특수 기호를 사용하여 가변 개수의 인수를 함수에 전달할 수 있습니다. 두 가지 특수 기호가 있습니다.

- **Python의 *args** (키워드가 아닌 인수)
- ****Python의 kwargs** (키워드 인수)

예 1: 가변 길이 비키워드 인수

```
def myFun(*argv):
    for arg in argv:
        print(arg)


myFun('Hello', 'Welcome', 'to', 'GeeksforGeeks')
```

예 2: 가변 길이 키워드 인수

```
def myFun(**kwargs):
    for key, value in kwargs.items():
        print("%s == %s" % (key, value))


myFun(first='Geeks', mid='for', last='Geeks')
```

문서 문자열

함수 다음 첫 번째 문자열을 문서 문자열(Document string) 또는 줄여서 Docstring 이라고 합니다 . 이는 함수의 기능을 설명하는 데 사용됩니다. 함수에서 docstring을 사용하는 것은 선택 사항이지만, 좋은 관행으로 간주됩니다.

아래 구문은 함수의 docstring을 출력하는 데 사용할 수 있습니다.

> ***구문:*** *print(함수_이름.__doc__)*

예: 함수에 Docstring 추가

```
def evenOdd(x):
    """Function to check if the number is even or odd"""

    if (x % 2 == 0):
        print("even")
    else:
        print("odd")


print(evenOdd.__doc__)
```

산출

함수 내의 Python 함수

다른 함수 안에 정의된 함수를 **내부 함수** 또는 **중첩 함수** 라고 합니다 . 중첩 함수는 해당 함수 범위를 포함하는 변수에 접근할 수 있습니다. 내부 함수는 함수 외부에서 발생하는 모든 일로부터 함수를 보호하기 위해 사용됩니다.

```
def f1():
    s = 'I love GeeksforGeeks'

    def f2():
        print(s)

    f2()

f1()
```

Python 함수의 반환 문

Python의 return **문** 은 함수를 종료하고 호출자에게 값을 반환하는 데 사용됩니다. 모든 **데이터 유형을 반환할 수 있으며, 여러 값이 쉼표로 구분된 경우 자동으로 튜플** 로 묶입니다 . 값을 지정하지 않으면 함수는 기본적으로 **None을 반환합니다.**

Syntax:

```
return [expression]
```

설명:

- **반환:** 함수를 종료하고 선택적으로 호출자에게 값을 전송합니다.
- **[표현식]:** 반환할 선택적 값, 생략하면 기본적으로 None이 지정됩니다.

예: Python 함수 반환 문

```
def square_value(num):
    """This function returns the square
    value of the entered number"""
    return num**2

print(square_value(2))
print(square_value(-4))
```

산출

참조로 전달 및 값으로 전달

한 가지 중요한 점은 Python에서 모든 변수 이름은 참조라는 것입니다. Python 함수에 변수를 전달하면 해당 객체에 대한 새로운 참조가 생성됩니다. Python에서 매개변수 전달은 Java에서 참조 전달과 동일합니다.

```
# Here x is a new reference to same list lst
def myFun(x):
    x[0] = 20

# Driver Code (Note that lst is modified
# after function call.
lst = [10, 11, 12, 13, 14, 15]
myFun(lst)
print(lst)
```

산출

```
[20, 11, 12, 13, 14, 15]
```

참조를 전달하고 수신된 참조를 다른 것으로 변경하면 전달된 매개변수와 수신된 매개변수 사이의 연결이 끊어집니다. 예를 들어, 아래 프로그램을 살펴보겠습니다.

```
def myFun(x):
    x = [20, 30, 40]


lst = [10, 11, 12, 13, 14, 15]
myFun(lst)
print(lst)
```

산출

```
[10, 11, 12, 13, 14, 15]
```

또 다른 예는 함수 내부에서 새 값을 할당하면 참조 링크가 끊어진다는 것을 보여줍니다.

```
def myFun(x):
    x = 20

x = 10
myFun(x)
print(x)
```

산출

```
10
```

파이썬 pass 문

Python 에서 Pass 문은 null 연산 또는 플레이스홀더입니다. 구문상 필수이지만 코드를 실행하고 싶지 않을 때 사용됩니다. 아무런 작업도 하지 않지만 프로그램의 구조를 유지할 수 있도록 해줍니다.

함수에서 Pass 키워드 사용 예

함수 의 키워드 Pass는 함수를 정의했지만 로직을 즉시 구현하고 싶지 않을 때 사용됩니다. 이를 통해 함수가 아직 아무런 동작을 수행하지 않더라도 구문적으로 유효하게 유지될 수 있습니다.

```
# Example of using pass in an empty function
def fun():
    pass  # Placeholder, no functionality yet

# Call the function
fun()
```

설명:

- fun() 함수는 정의되어 있지만 pass 문을 포함하고 있어 호출 시 아무 작업도 수행되지 않습니다.
- 프로그램은 오류 없이 계속 실행되고, 함수를 호출한 후 메시지가 인쇄됩니다.

조건문에서 pass 사용

조건문 (if, elif 또는 else 블록 등) 에서 pass 문은 특정 조건에 대해 아무런 동작도 실행하지 않을 때 사용됩니다.

```
x = 10

if x > 5:
    pass  # Placeholder for future logic
else:
    print("x is 5 or less")
```

산출

설명:

- x > 5일 때 pass 문이 실행되어 아무 일도 일어나지 않습니다.
- x <= 5이면 else 블록이 실행되어 "x는 5 이하입니다"라는 문장이 출력됩니다.
- pass를 사용하면 if 블록이 현재 비어 있더라도 코드를 오류 없이 컴파일하고 실행할 수 있습니다.

루프에서 패스 사용

루프 (for나 while 등) 에서 pass를 사용하면 반복 중에 아무런 작업도 필요하지 않음을 나타낼 수 있습니다.

```
for i in range(5):
    if i == 3:
        pass  # Do nothing when i is 3
    else:
        print(i)
```

산출

```
0
1
2
4
```

설명:

- 루프는 0에서 4까지의 숫자를 반복합니다.
- i == 3일 때, pass 문이 실행되며, 이는 해당 반복에 대해 아무런 작업도 수행되지 않음을 의미합니다.
- i의 다른 값에 대해서는 숫자가 출력됩니다.

Python의 전역 변수와 지역 변수

Python 에서 **전역 변수는** 함수 외부에서 선언되며 함수 내부를 포함하여 프로그램 어디에서나 접근할 수 있습니다. 반면, 지역 **변수는** 함수 내에서 생성되며 해당 함수가 실행되는 동안에만 접근할 수 있습니다. 즉, 지역 변수는 정의된 함수 내부에서만 존재하며 함수 외부에서는 사용할 수 없습니다. 각 변수를 자세히 살펴보겠습니다.

파이썬 지역 변수

지역 변수는 함수 내에서 생성되어 함수가 실행되는 동안만 존재합니다. 함수 외부에서는 접근할 수 없습니다.

예제 1: 이 예제에서는 함수 내에서 로컬 변수를 생성하고 접근합니다.

```
def greet():
    msg = "Hello from inside the function!"
    print(msg)

greet()
```

산출

설명: 지역 변수 **msg를 사용하여 greet()를** 정의하고 출력합니다. msg는 함수 실행 중에만 존재하므로 함수 내부에서 접근합니다. **greet()를** 호출하면 메시지가 표시됩니다.

예제 2 : 이 예제에서는 함수 내부에 로컬 변수를 생성한 다음 함수 외부에서 해당 변수에 접근하려고 시도하는데, 이로 인해 오류가 발생합니다.

```
def greet():
    msg = "Hello!"
    print("Inside function:", msg)

greet()
print("Outside function:", msg)
```

산출

```
Hangup(SIGHUP)
추적(최근 호출이 마지막):
  파일 "/home/guest/sandbox/Solution.py", 6번째 줄, <모듈>에서
    print("함수 외부:", msg)
                       ^^^
NameError: 이름 'msg'가 정의되지 않았습니다.
```

설명: msg는 greet() 내부의 지역 변수이므로 해당 함수에서만 접근할 수 있습니다. msg를 출력하면 전역 변수가 아니므로 오류가 발생합니다.

파이썬 전역 변수

전역 변수는 모든 함수 밖에서 정의됩니다. 함수 내부를 포함하여 프로그램의 어느 부분에서나 접근하고 사용할 수 있습니다.

예제 1: 이 예제에서는 전역 변수를 생성한 다음 함수 내부와 외부에서 모두 해당 변수에 액세스합니다.

```
msg = "Python is awesome!"

def display():
    print("Inside function:", msg)

display()
print("Outside function:", msg)
```

산출

설명: msg는 display() 함수 내부와 외부 모두에서 접근 가능한 전역 변수입니다 . **display()를** 호출하면 전역 **msg가** 출력되고 , 함수 외부에서 **msg를 출력하면 예상대로 작동합니다.**

예제 2: 이 예제에서는 전역 변수를 만든 다음 함수 내부와 외부에서 모두 사용합니다.

```
def fun():
    print("Inside Function", s)

# Global scope
s = "I love Geeksforgeeks"
fun()
print("Outside Function", s)
```

산출

설명: s는 fun() 함수 내부에서 접근하고 출력하는 전역 변수입니다 . **fun() 함수** 를 호출하는 것과 함수 외부에서 s를 출력하는 것은 모두 동일한 전역 값을 나타냅니다.

> ***참고:*** *로컬 변수가 없으므로 전역 변수의 값이 사용되지만 로컬 변수와 전역 변수의 이름이 같아야 합니다.*

Python에서 지역 변수와 전역 변수를 사용하는 이유는 무엇입니까?

변수가 전역 변수와 지역 변수 모두에 같은 이름으로 정의된 경우, 지역 변수는 함수 내에서 전역 변수를 가립니다. 변수를 명시적으로 전역 변수로 선언하지 않는 한, 지역 변수의 변경 사항

은 전역 변수에 영향을 미치지 않습니다.

```
def fun():
    s = "Me too."
    print(s)

s = "I love Geeksforgeeks"
fun()
print(s)
```

산출

설명: fun() 내부에서 **s는** "Me too."로 설정된 지역 변수이며, 해당 값을 출력합니다. 외부에서는 전역 변수 s **가** "I love Geeksforgeeks"로 유지되므로, 나중에 **s를** 출력하면 전역 변수 값이 표시됩니다.

함수 내에서 전역 변수를 수정하려고 하면 어떻게 될까?

함수 내에서 전역 변수를 전역으로 선언하지 않고 변경하려고 하면 오류가 발생합니다.

```
def fun():
    s += 'GFG'
    print("Inside Function", s)

s = "I love Geeksforgeeks"
fun()
```

산출

```
UnboundLocalError : 할당 전에 로컬 변수 's'가 참조되었습니다.
```

설명: fun() 함수는 s를 전역 변수로 선언하지 않고 수정하려고 합니다 . 따라서 Python은 **s를** 지역 변수로 취급하지만, 대입 전에 사용되어 오류가 발생합니다. **fun() 함수 내에서 s를** 전역 변수로 선언하면 이 문제가 해결됩니다.

함수 내에서 전역 변수 수정

함수 내에서 전역 변수를 수정하려면 global 키워드를 사용하여 Python에 전역 변수를 사용하고 싶다는 것을 명시적으로 알려야 합니다 .

```
def fun():
    global s
    s += ' GFG'    # Modify the global variable
    print(s)
    s = "Look for Geeksforgeeks Python Section"
    print(s)

s = "Python is great!"
fun()
print(s)
```

산출

설명: fun() 함수 내부에서 global 키워드를 사용하면 Python이 전역 변수 **s를** 직접 수정할 수 있습니다. 이 함수는 먼저 "Python is great!"에 'GFG'를 추가한 다음, s를 "Look for Geeksforgeeks Python Section"에 다시 할당합니다.

예제 2: 이 예제는 Python이 같은 이름을 가진 전역 변수와 지역 변수를 처리하는 방법과 **global 키워드가 변수의** 동작에 어떤 영향을 미치는지 보여줍니다.

```
a = 1  # Global variable

def f():
    print('f():', a)  # Uses global a

def g():
    a = 2  # Local variable shadows global
    print('g():', a)

def h():
    global a
    a = 3  # Modifies global a
    print('h():', a)

print('global:', a)
f()
print('global:', a)
g()
print('global:', a)
h()
print('global:', a)
```

산출

```
글로벌: 1
f(): 1
글로벌: 1
g(): 2
글로벌: 1
h(): 3
글로벌: 3
```

설명:

- **f()는** 전역 **a를** 변경하지 않고 출력합니다.
- **g()는** 전역 a를 변경하지 않고 그대로 두고 전역 **a를** 가리는 지역 **a** 를 생성합니다 .
- **h()는** global을 사용하여 전역 **a를 수정합니다.**
- **h()** 만 전역 변수를 변경하고 **f()** 와 **g()는** 변경하지 않습니다.

파이썬에서 지역 변수와 전역 변수는 범위와 수명이 다르기 때문에 이를 이해하는 것이 중요합니다. 지역 변수는 함수 내부에 존재하고 전역 변수는 어디에서나 접근 가능합니다. 이러한 지식은 버그를 방지하고 더 깔끔한 코드를 작성하는 데 도움이 됩니다. 아래 비교표를 참고하여 명확하게 이해하세요.

비교 기준	전역 변수	지역 변수
정의	함수 외부에서 선언됨	함수 내에서 선언됨
일생	프로그램 실행이 시작되면 생성되고 프로그램이 종료되면 사라집니다.	함수가 실행을 시작할 때 생성되고 함수가 종료되면 손실됩니다.
데이터 공유	데이터 공유 제공	데이터 공유를 제공하지 않습니다
범위	코드 전체에서 접근 가능	함수 내부에서만 접근 가능
필요한 매개변수	매개변수 전달은 필요하지 않습니다	매개변수 전달이 필요합니다
저장	컴파일러가 선택한 고정 위치	스택에 보관됩니다
값	값이 변경되면 코드 전체에 반영됩니다.	변수가 변경되면 프로그램의 다른 기능에 영향을 미치지 않습니다.

Python의 *args와 **kwargs

Python에서는 *args와 **kwargs를 사용하여 함수가 임의의 개수의 인수를 받을 수 있도록 합니다. 이러한 기능은 다양한 개수의 입력을 처리해야 하는 함수를 설계할 때 뛰어난 유연성을 제공합니다.

예:

```
# *args example
def fun(*args):
    return sum(args)

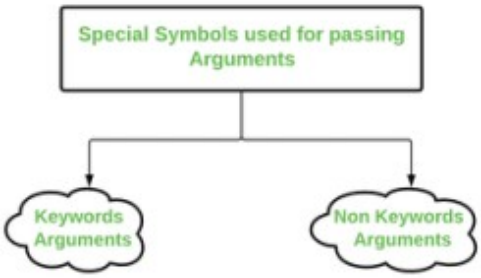
print(fun(1, 2, 3, 4))
print(fun(5, 10, 15))

# **kwargs example
def fun(**kwargs):
    for k, val in kwargs.items():
        print(k, val)

fun(a=1, b=2, c=3)
```

산출

여러 인수를 전달하는 데는 두 가지 특수 기호가 있습니다.



Python에서 인수를 전달하는 데 사용되는 특수 기호:

- *args (키워드가 아닌 인수)
- **kwargs(키워드 인수)

참고: *"함수에 전달해야 할 인수의 개수에 대해 의심이 들 때, 우리는 "와일드카드" 또는 "*" 표기법을 *args 또는 **kwargs와 같이 함수의 인수로 사용합니다."*

파이썬 *args

함수 정의에서 특수 구문 ***args는*** 가변 개수의 인수를 함수에 전달하는 데 사용됩니다. 키워드가 지정되지 않은 가변 길이 인수 목록을 전달하는 데 사용됩니다. 이러한 인수는 **튜플로 그룹화되어** 함수가 여러 개의 입력을 처리할 수 있도록 합니다.

예를 들어 multiply() 함수는 *args를 사용하여 원하는 개수의 입력을 받아 곱할 수 있습니다. *는 변수를 반복 가능하게 만들어 루프를 돌거나 map() 및 filter()와 같은 함수와 함께 사용할 수 있습니다.

예시 1:

가변 개수의 인수에 대한 *args를 설명하는 Python 프로그램

```
def myFun(*argv):
    for arg in argv:
        print(arg)

myFun('Hello', 'Welcome', 'to', 'GeeksforGeeks')
```

산출

예 2:

첫 번째 추가 인수를 사용하여 *args를 설명하는 Python 프로그램입니다.

```
def fun(arg1, *argv):
    print("First argument :", arg1)
    for arg in argv:
        print("Argument *argv :", arg)

fun('Hello', 'Welcome', 'to', 'GeeksforGeeks')
```

산출

파이썬 **kwargs

함수 정의에서 특수 구문 ***kwargs는** *가변 길이 인수 목록을 전달하는 데 사용됩니다. kwargs* 라는 이름 에 이중 별표 **를 붙입니다.

- 키워드 인수는 함수에 변수를 전달할 때 변수에 이름을 제공하는 인수입니다.
- 이는 함수에 전달된 모든 추가 키워드 인수를 수집하여 **사전에 저장합니다** .

예시 1:

```
def fun(**kwargs):
    for k, val in kwargs.items():
        print("%s == %s" % (k, val))

# Driver code
fun(s1='Geeks', s2='for', s3='Geeks')
```

산출

s1='Geeks'의 경우, s1은 키이고 'Geeks'는 값입니다. 간단히 말해, 우리가 할당하는 것이 값이고, 할당하는 대상이 키입니다.

예 2:

```
def fun(arg1, **kwargs):
    for k, val in kwargs.items():
        print("%s == %s" % (k, val))

# Driver code
fun("Hi", s1='Geeks', s2='for', s3='Geeks')
```

산출

*args와 **kwargs를 모두 사용

동일한 함수에서 *args와 **kwargs를 모두 사용하여 위치 인수와 키워드 인수를 혼합하여 사용할 수 있습니다.

예:

```
def fun(*args, **kwargs):
    print("Positional arguments:", args)
```

```
    print("Keyword arguments:", kwargs)

fun(1, 2, 3, a=4, b=5)
```

산출

이 예제에서 fun은 위치 인수와 키워드 인수를 모두 처리할 수 있습니다. args 매개변수는 위치 인수를 튜플로 수집하고, kwargs 매개변수는 키워드 인수를 딕셔너리로 수집합니다.

Python의 일급 함수

일급 함수는 함수를 일급 시민으로 취급하는 개념입니다. 파이썬은 함수를 일급 시민으로 취급함으로써 더욱 추상적이고 재사용 가능하며 모듈화된 코드를 작성할 수 있도록 합니다. 즉, 이러한 언어에서 함수는 다른 변수와 마찬가지로 취급됩니다. 다른 함수에 인수로 전달되거나, 다른 함수에서 값으로 반환되거나, 변수에 할당되거나, 데이터 구조에 저장될 수 있습니다.

일급 함수의 특성

- **변수에 할당** : 변수에 함수를 할당할 수 있습니다.
- **인수로 전달** : 함수를 다른 함수에 인수로 전달할 수 있습니다.
- **함수에서 반환됨** : 함수는 다른 함수를 반환할 수 있습니다.
- **데이터 구조에 저장**: 함수는 목록, 사전 등의 데이터 구조에 저장될 수 있습니다.

변수에 함수 할당

변수에 함수를 할당하고 해당 변수를 사용하여 함수를 호출할 수 있습니다.

예:

```
def msg(name):
    return f"Hello, {name}!"

# Assigning the function to a variable
f = msg

# Calling the function using the variable
print(f("Anurag"))
```

설명:

- 함수 greet가 변수 f에 할당됩니다.
- f는 함수를 호출하는 데 사용되며, 이는 함수가 다른 변수와 마찬가지로 취급될 수 있음을 보여줍니다.

인수로 함수 전달

함수는 다른 함수에 인수로 전달되어 고차 함수를 사용할 수 있습니다 .

예:

```
def msg(name):
    return f"Hello, {name}!"

def fun1(fun2, name):
    return fun2(name)

# Passing the greet function as an argument
print(fun1(msg, "Bob"))
```

설명:

- fun1 함수는 다른 함수 fun2와 이름을 인수로 받습니다.
- msg 함수는 fun1에 전달되고, fun1은 지정된 이름으로 greet을 호출합니다.

다른 함수에서 함수 반환

함수는 다른 함수를 반환할 수 있으므로 함수 팩토리를 만들 수 있습니다.

예:

```
def fun1(msg):
    def fun2():
        return f"Message: {msg}"
    return fun2

# Getting the inner function
func = fun1("Hello, World!")
print(func())
```

설명:

- fun1은 fun2를 정의하고 반환합니다.
- func는 반환된 fun2를 저장하는데, 이 fun2는 나중에 호출될 수 있습니다.

데이터 구조에 함수 저장

예:

```
def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

# Storing functions in a dictionary
d = {
```

```
    "add": add,
    "subtract": subtract
}

# Calling functions from the dictionary
print(d["add"](5, 3))
print(d["subtract"](5, 3))
```

설명:

- add와 subtract 함수는 사전에 저장됩니다.
- 함수는 각각의 키를 사용하여 사전에서 접근하고 호출됩니다.

파이썬 내부 함수

파이썬에서 다른 함수 안에 있는 **함수를 내부 함수** 또는 **중첩 함수** 라고 합니다 . 내부 함수는 코드 구성, 가독성 향상, 캡슐화 유지에 도움이 됩니다. 또한, 외부 함수의 변수에 접근할 수 있으므로 클로저 와 함수 데코레이터를 구현하는 데 유용합니다 .

예:

```
def fun1(msg): # outer function

    def fun2(): # inner function
        print(msg)
    fun2()

fun1("Hello")
```

설명: 여기서 **fun2() 는 fun1()** 내부에 정의되어 있으며, 둘러싼 범위에서 **msg** 변수에 접근합니다 .

왜 내부 함수를 사용해야 하나요?

내부 기능은 다음과 같은 여러 가지 이점을 제공합니다.

- **캡슐화: 캡슐화** 는 내부 논리를 외부 접근으로부터 숨기는 데 도움이 됩니다.
- **코드 구성:** 관련 기능을 그룹화하여 코드를 더 깔끔하게 만듭니다.
- **둘러싼 범위에 대한 접근:** 내부 함수는 외부 함수의 변수에 접근할 수 있습니다.
- **클로저: 클로저를** 사용하면 함수가 실행 후에도 해당 함수의 상태를 유지할 수 있습니다.

내부 함수의 변수 범위

내부 함수는 자신을 둘러싼(외부) 함수의 변수에 접근할 수 있지만, 이를 수정하려면 특별한 처리가 필요합니다. 이는 Python의 변수 범위 규칙인 LEGB (Local, Enclosing, Global, Built-in)를 따릅니다.

예제 1 : 로컬 변수 액세스

```
def fun1(): # outer function
    msg = "Geeks for geeks"

    def fun2(): # inner function
```

```
        print(msg)  # accessing outer function's variable

    fun2()
fun1()
```

설명: fun1()은 지역 변수 **msg 와 msg를** 출력하는 내부 함수 **fun2()를** 정의합니다 . 어휘적 유효 범위(lexical scoping) 때문에 **fun2()는 fun1()** 에서 **msg** 에 접근합니다 . **fun1()을** 호출하면 **fun2()가** 호출되어 메시지를 출력합니다.

예제 2: nonlocal을 사용하여 변수 수정

```
def fun1(): # outer function
    a = 45

    def fun2(): # inner function
        nonlocal a  # allows modification of `a` from fun1
        a=54
        print(a)

    fun2()
    print(a)

fun1()
```

설명: nonlocal 키워드를 사용하면 **fun2() 함수가 fun1()** 함수의 변수 **a를** 수정할 수 있습니다. nonlocal 키워드가 없으면 **fun2() 함수 내부의 a는 fun1()** 함수의 a를 수정하는 대신 새로운 지역 변수로 처리됩니다 .

예제 3 : 내부 함수의 클로저

```
def fun1(a): # outer function

    def fun2(): # inner function
        print(a)
    return fun2  # returning function without parentheses

closure_func = fun1("Hello, Closure!")
closure_func()  # inner function remembers 'a'
```

설명: fun1()이 실행을 완료한 후에도 반환된 **fun2() 함수는 a** 에 대한 액세스를 유지하여 클로저를 보여줍니다.

내부 함수 사용을 위한 모범 사례

내부 함수는 강력하지만 코드 가독성, 효율성, 그리고 유지 관리 용이성을 유지하기 위해 현명하게 사용해야 합니다. 다음은 몇 가지 모범 사례입니다.

- **꼭 필요한 경우에만 내부 함수를 사용하세요.** 과도한 중첩은 가독성을 떨어뜨릴 수 있으므로 피하세요.
- **클로저를 현명하게 사용하세요.** 캡처된 변수가 적절하게 관리되어 의도치 않은 부작용이 발생하지 않도록 하세요.
- **전역 변수보다 nonlocal 변수를 선호합니다.** 외부 함수 변수를 수정하는 경우 global 대신 nonlocal을 사용합니다.
- **데코레이터에서 내부 함수를 사용합니다.** 이는 일반적이고 효과적인 사용 사례입니다.

파이썬 클로저

파이썬 에서 클로저는 함수가 렉시컬 스코프 밖에서 실행될 때에도 함수가 렉시컬 스코프의 변수를 기억하고 접근할 수 있도록 하는 강력한 개념입니다. 클로저는 중첩 함수와 밀접한 관련이 있으며 함수형 프로그래밍, 이벤트 처리 및 콜백에 일반적으로 사용됩니다.

클로저는 한 함수 (내부 함수)가 다른 함수(외부 함수) 내에 정의되고, 내부 함수가 외부 함수의 변수를 참조할 때 생성됩니다. 클로저는 전역 변수를 사용하지 않고도 함수가 여러 호출에서 상태를 유지해야 할 때 유용합니다 .

파이썬 클로저의 기본 예

클로저가 어떻게 작동하는지 이해하기 위해 간단한 예를 살펴보겠습니다.

```
def fun1(x):

    # This is the outer function that takes an argument 'x'
    def fun2(y):

        # This is the inner function that takes an argument 'y'
        return x + y  # 'x' is captured from the outer function

    return fun2  # Returning the inner function as a closure

# Create a closure by calling outer_function
closure = fun1(10)

# Now, we can use the closure, which "remembers" the value of 'x' as 10
print(closure(5))
```

산출

```
15
```

설명:

- **외부 함수(fun1):** 인수 x를 받고 fun2를 정의합니다. fun2는 x와 다른 인수 y를 사용하여 계산을 수행합니다.
- **내부 함수(fun2):** 이 함수는 fun1에서 반환되므로 클로저입니다. fun1의 실행이 완료된 후에도 x의 값을 "기억"합니다.
- **클로저 생성 및 사용:** fun1(10)을 호출하면 x가 10으로 설정된 fun2를 반환합니다. 반환된 fun2(closure)는 closure 변수에 저장됩니다. closure(5)를 호출하면 기억된 x 값(10)과 전달된 인수 y(5)를 사용하여 10 + 5 = 15의 합을 계산합니다.

파이썬 클로저를 자세히 살펴보겠습니다.

Python 클로저의 예:

```
def fun(a):
    # Outer function that remembers the value of 'a'
    def adder(b):
        # Inner function that adds 'b' to 'a'
        return a + b
    return adder  # Returns the closure

# Create a closure that adds 10 to any number
val = fun(10)

# Use the closure
print(val(5))
print(val(20))
```

산출

```
15
30
```

이 예에서는:

- fun(10)은 값 10을 기억하고 클로저에 전달된 모든 숫자에 그 값을 더하는 클로저를 생성합니다.

클로저는 내부적으로 어떻게 작동하나요?

클로저가 생성될 때, Python은 클로저가 정의된 환경(둘레 범위 내의 변수)에 대한 참조를 내부적으로 저장합니다. 이를 통해 외부 함수가 완료된 후에도 내부 함수가 해당 변수에 접근할 수 있습니다.

간단히 말해서, 클로저는 주변 스코프의 값을 "캡처"하여 나중에 사용할 수 있도록 보관합니다. 이를 통해 클로저는 환경의 값을 기억할 수 있습니다.

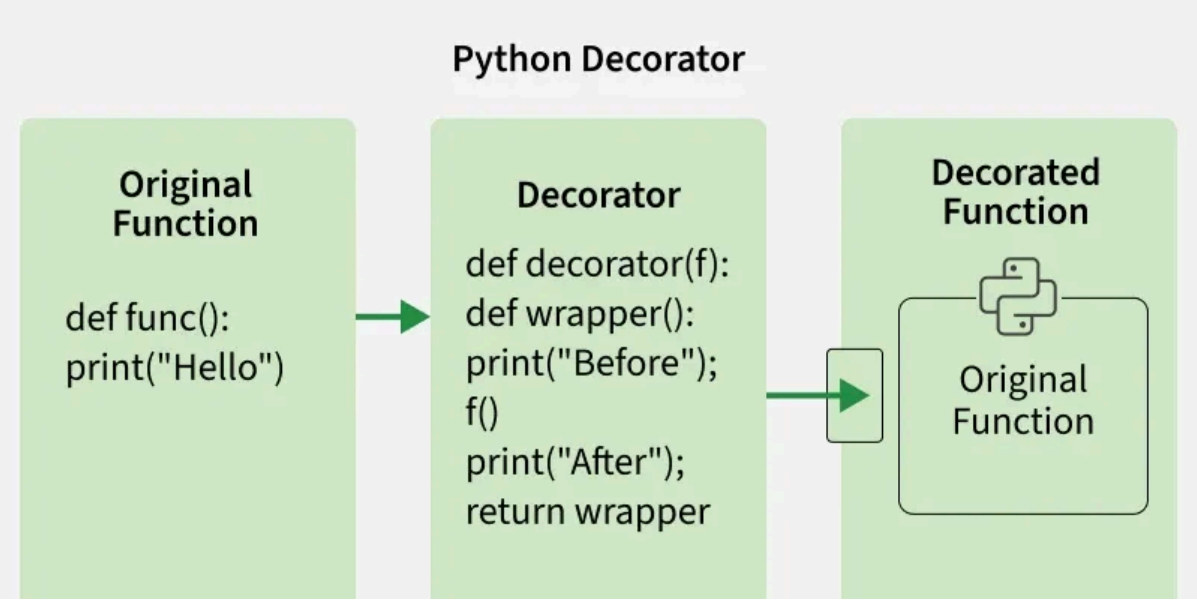
클로저 사용

- **캡슐화:** 클로저는 기능을 캡슐화하는 데 도움이 됩니다. 내부 함수는 외부 함수의 변수에 접근할 수 있지만, 해당 변수는 외부에서는 숨겨져 있습니다.
- **상태 유지:** 클로저는 여러 함수 호출에서 상태를 유지할 수 있습니다. 이는 카운터, 어큐뮬레이터 또는 다양한 동작을 가진 함수를 생성하는 함수 팩토리를 만들 때 특히 유용합니다.
- **함수형 프로그래밍:** 클로저는 함수형 프로그래밍의 핵심 기능입니다. 클로저를 사용하면 새로운 동작을 동적으로 생성하여 더욱 유연하고 모듈화된 코드를 작성할 수 있습니다.

파이썬의 데코레이터

Python 에서 데코레이터는 실제 코드를 변경하지 않고도 함수나 메서드의 동작을 수정하거나 확장하는 유연한 방법입니다.

- 데코레이터는 본질적으로 다른 함수를 인수로 받아서 향상된 기능을 가진 새로운 함수를 반환하는 함수 입니다.
- 데코레이터는 로깅, 인증, 기억과 같은 시나리오에서 자주 사용되며, 깔끔하고 재사용 가능한 방식으로 기존 함수나 메서드에 새로운 기능을 추가할 수 있습니다.



예: 간단한 Python 데코레이터가 함수 호출 전후에 동작을 추가하는 예를 살펴보겠습니다.

```
def decorator(func):
    def wrapper():
        print("Before calling the function.")
        func()
        print("After calling the function.")
    return wrapper

@decorator # Applying the decorator to a function
def greet():
    print("Hello, World!")
greet()
```

산출

설명:

- **데코레이터는** greet 함수를 인수로 받습니다.
- **이 함수는 먼저 메시지를 출력하고 greet()를** 호출한 다음 다른 메시지를 출력하는 새로운 함수(래퍼)를 반환합니다 .
- **@decorator 구문은 greet = decorator(greet)** 의 약어입니다 .

매개변수가 있는 데코레이터

데코레이터는 종종 인수가 있는 함수를 사용해야 합니다. 래퍼가 원하는 개수의 인수를 받을 수 있도록 ***args** 와 ****kwargs를 사용합니다.**

예: 함수 호출 전후에 기능을 추가하는 데코레이터의 예를 살펴보겠습니다.

```
def decorator_name(func):
    def wrapper(*args, **kwargs):
        print("Before execution")
        result = func(*args, **kwargs)
        print("After execution")
        return result
    return wrapper

@decorator_name
def add(a, b):
    return a + b

print(add(5, 3))
```

산출

매개변수 설명

1. 데코레이터 이름(함수):

- **decorator_name:** 데코레이터 함수의 이름입니다.
- **func:** 이 매개변수는 데코레이터로 데코레이팅되는 함수를 나타냅니다. 데코레이터를 사용하면 데코레이팅된 함수가 이 매개변수로 전달됩니다.

2. 래퍼(*args, **kwargs):

- **래퍼(wrapper):** 데코레이터 내부에 중첩된 함수입니다. 기존 함수를 감싸서 추가 기능을 제공합니다.
- ***args:** 이것은 데코레이팅된 함수에 전달된 모든 위치 인수를 튜플로 수집합니다.
- ****kwargs:** 이것은 데코레이팅된 함수에 전달된 모든 키워드 인수를 사전으로 수집합니

다.

- 래퍼 함수를 사용하면 데코레이터가 원하는 개수와 유형의 인수를 갖는 함수를 처리할 수 있습니다.

3. @데코레이터_이름:

- 이 구문은 데코레이터를 **function_to_decorate** 함수에 적용합니다.
- 이는 **function_to_decorate = decorator_name(function_to_decorate)** 라고 쓰는 것과 같습니다 .

일급 객체로서의 함수

Python에서 함수는 일급 객체 입니다 . 즉, 다른 객체(정수, 문자열, 리스트 등)처럼 취급할 수 있습니다. 이를 통해 함수를 **변수에 할당하고** , **인수로 전달하고** , **다른 함수에서 반환하고** , 데코레이터를 포함한 유연한 프로그래밍 패턴을 지원하는 **데이터 구조에 저장할 수 있습니다**

예: 이 코드는 Python에서 일급 객체로서 함수의 네 가지 속성을 모두 보여줍니다.

```python
# Assigning a function to a variable
def greet(n):
    return f"Hello, {n}!"
say_hi = greet  # Assign the greet function to say_hi
print(say_hi("Alice"))

# Passing a function as an argument
def apply(f, v):
    return f(v)
res = apply(say_hi, "Bob")
print(res)

# Returning a function from another function
def make_mult(f):
    def mult(x):
        return x * f
    return mult
dbl = make_mult(2)
print(dbl(5))
```

산출

설명:

- **greet() 는 say_hi** 변수 에 할당되어 "Alice"에게 인사말을 출력하는 데 사용됩니다.
- **apply()는** 함수와 값을 인수로 받고, 함수를 값에 적용하고 결과를 반환합니다.
- **apply는 say_hi** 와 "Bob"을 전달하여 "Bob"에 대한 인사말을 출력하는 방식으로 시연됩니다 .
- **make_mult()는** 주어진 요인에 따라 승수 함수를 생성합니다.

데코레이터에서 일급 함수의 역할

- 데코레이터는 함수를 입력으로 받아서 동작을 수정하거나 향상시킵니다.
- 이들은 원래 함수를 래핑하고 실행 전이나 후에 동작을 추가하는 새로운 함수를 반환합니다.
- 동일한 이름에 할당되면 원래 함수는 데코레이팅된 함수로 대체됩니다.

고차 함수

고차 함수 는 하나 이상의 함수를 인수로 받거나, 결과로 함수를 반환하거나, 둘 다 수행하는 함수입니다. 본질적으로 고차 함수는 다른 함수에 대해 동작하는 함수입니다. 이는 함수형 프로그래밍에서 강력한 개념이며, 데코레이터의 작동 방식을 이해하는 데 중요한 요소입니다.

고차 함수의 주요 속성

- **함수를 인수로 받는 경우** : 고차 함수는 다른 함수를 매개변수로 받을 수 있습니다.
- **함수 반환** : 고차 함수는 나중에 호출될 수 있는 새로운 함수를 반환할 수 있습니다.

예: 이 코드는 다른 함수를 인수로 받아서 주어진 값에 적용하는 고차 함수를 보여줍니다.

```
# A higher-order function that takes another function as an argument
def fun(f, x):
    return f(x)

# A simple function to pass
def square(x):
    return x * x
res = fun(square, 5) # Using apply_function to apply the square function
print(res)
```

산출

```
25
```

설명: fun은 다른 함수 f를 인수로 받아서 값 **x** 에 적용하기 때문에 고차 함수입니다 .

데코레이터에서 고차 함수의 역할

데코레이터는 함수를 받아서 수정하고 확장되거나 변경된 동작을 갖는 새 함수를 반환하는 고차 함수입니다.

1. 함수 데코레이터

Python에서 가장 일반적인 데코레이터로, 원래 함수가 실행되기 전이나 실행된 후에 추가 동작을 추가하여 함수를 래핑하고 향상시키는 데 사용됩니다.

예: 이 예에서 데코레이터는 래핑된 함수를 실행하기 전과 후에 메시지를 출력합니다.

```
def simple_decorator(func):
    def wrapper():
        print("Before calling the function.")
        func()
        print("After calling the function.")
    return wrapper

@simple_decorator
def greet():
    print("Hello, World!")
greet()
```

설명:

- **simple_decorator(func):** 이 데코레이터는 greet 함수를 인수(func)로 받고 원래 함수를 호출하기 전과 후에 일부 기능을 추가하는 새로운 함수(래퍼)를 반환합니다.
- **@simple_decorator:** 데코레이터 구문입니다. greet 함수에 simple_decorator를 적용합니다.
- **greet() 호출:** greet()가 호출되면 원래 함수를 실행하는 것이 아니라 먼저 래퍼 함수에서 추가된 동작을 실행합니다.

2. 메서드 데코레이터

클래스 내부 메서드에 사용되는 특수 데코레이터입니다. 함수 데코레이터처럼 작동하지만 인스턴스 메서드의 self 매개변수를 처리합니다.

예: 여기에서는 데코레이터가 메서드가 실행되기 전과 후에 메시지를 출력하면서 self 인수를 올바르게 처리합니다.

```
def method_decorator(func):
    def wrapper(self, *args, **kwargs):
        print("Before method execution")
        res = func(self, *args, **kwargs)
        print("After method execution")
        return res
    return wrapper

class MyClass:
    @method_decorator
    def say_hello(self):
        print("Hello!")
obj = MyClass()
obj.say_hello()
```

설명:

- **method_decorator(func):** 데코레이터는 메서드(say_hello)를 인수(func)로 받습니다. 원래 메서드를 호출하기 전과 호출한 후에 동작을 추가하는 래퍼 함수를 반환합니다.
- **wrapper(self, *args, **kwargs):** wrapper는 인스턴스의 메서드이므로 self를 허용해야 합니다. self는 클래스의 인스턴스이고 *args와 **kwargs를 사용하면 필요한 경우 다른 인수를 전달할 수 있습니다.
- **@method_decorator:** 이것은 MyClass의 say_hello 메서드에 method_decorator를 적용합니다.
- **obj.say_hello() 호출:** say_hello 메서드는 이제 추가 동작으로 래핑되었습니다.

3. 클래스 데코레이터

클래스 데코레이터는 클래스의 동작을 수정하거나 향상시키는 데 사용됩니다. 함수 데코레이터와 마찬가지로 클래스 데코레이터는 클래스 정의에 적용됩니다. 클래스 데코레이터는 클래스를 인수로 받고 수정된 버전의 클래스를 반환합니다.

예: 이 코드는 클래스에 class_name 속성을 추가하여 클래스 이름을 저장하는 클래스 데코레이터를 보여줍니다 .

```
def fun(cls):
    cls.class_name = cls.__name__
    return cls

@fun
class Person:
    pass
print(Person.class_name)
```

설명:

- **add_class_name(cls):** 이 데코레이터는 클래스 cls에 새 속성인 class_name을 추가합니다. class_name의 값은 클래스 이름(cls.__name__)으로 설정됩니다.
- **@add_class_name:** 이것은 Person 클래스에 add_class_name 데코레이터를 적용합니다.

Python에서 매개변수가 있는 데코레이터

데코레이터는 프로그래머가 함수나 클래스의 동작을 수정할 수 있도록 해주는 Python의 매우 강력하고 유용한 도구입니다. 이 글에서는 여러 예제를 통해 매개변수가 있는 데코레이터에 대해 알아보겠습니다 . Python ***함수*** 는 일급 객체(First Class Citizen)이므로 객체와 유사하게 취급할 수 있습니다.

- 함수는 변수에 할당될 수 있습니다. 즉, 참조될 수 있습니다.
- 함수는 다른 함수의 인수로 전달될 수 있습니다.
- 함수는 함수로부터 반환될 수 있습니다.

매개변수가 있는 데코레이터는 일반 데코레이터와 비슷합니다.

매개변수가 있는 데코레이터의 구문:

```
@decorator(params)
def func_name():
    ''' Function implementation'''
```

위의 코드는 다음과 같습니다.

```
def func_name():
    ''' Function implementation'''

func_name = (decorator(params))(func_name)
"""
```

실행은 왼쪽에서 오른쪽으로 시작하며, **데코레이터(params)** 가 호출되어 함수 객체 **fun_obj를** 반환합니다 . fun_obj를 사용하여 **fun_obj(fun_name) 을 호출합니다. 내부 함수 내부에서 필요한 연산이 수행되고 실제 함수 참조가 반환되어** func_name 에 할당됩니다 . 이제 **func_name()을** 사용하여 데코레이터가 적용된 함수를 호출할 수 있습니다.

먼저 다음 사항을 살펴보세요.

```
# Python code to illustrate
# Decorators basic in Python
def decorator_fun(func):
    print("Inside decorator")
    def inner(*args,**kwargs):
        print("Inside inner function")
        print("Decorated the function")
        # do operations with func
        func()
    return inner()
@decorator_fun
def func_to():
    print("Inside actual function")
func_to
```

산출

또 다른 방법:

```
# Python code to illustrate
# Decorators with parameters in Python

def decorator_fun(func):
  print("Inside decorator")

  def inner(*args, **kwargs):
    print("Inside inner function")
    print("Decorated the function")

    func()

  return inner


def func_to():
    print("Inside actual function")

# another way of using decorators
decorator_fun(func_to)()
```

다른 예를 살펴보겠습니다.

예시 1:

코드 설명은 다음과 같습니다.

- `decorator(like)`: 데코레이터 함수는 이제 매개변수를 직접 허용합니다
- `inner(func)`: 이 함수는 실제 함수를 래핑 `func`하고 해당 범위 내에서 매개변수를 사용합니다
- `wrapper()`: `wrapper`함수는 실제를 호출하는 데 사용되고 `func`, 함수에서 반환됩니다 `inner`.

```python
# Python code to illustrate
# Decorators with parameters in Python
def decorator(like):
    print("Inside decorator")

    def inner(func):
        # code functionality here
        print("Inside inner function")
        print("I like", like)

        def wrapper():
            func()

        return wrapper

    # returning inner function
    return inner

@decorator(like="geeksforgeeks")
def my_func():
    print("Inside actual function")

# Call the decorated function
my_func()
```

산출

예 2:

코드 설명은 다음과 같습니다.

- `decorator_func(x, y)`: 이것은 데코레이터 팩토리입니다. 두 개의 매개변수 `x`와 를 받아서 `y`데코레이터를 반환합니다.
- `Inner(func)`: 이것은 에서 반환된 실제 데코레이터입니다 `decorator_func`. 함수를 받아서 그 함수를 중심으로 `func`새로운 함수를 정의합니다 `wrapper`.
- `wrapper(*args, **kwargs)`: 이 함수는 데코레이팅된 함수가 호출될 때 실행됩니다. 원래 함수를 호출하기 전에 일부 메시지와 `x`와 의 합계를 출력합니다 .`yfunc`
- `return wrapper`: `wrapper`함수는 함수에 의해 반환됩니다 `Inner`.
- `return Inner`: `Inner`데코레이터는 .에 의해 반환됩니다 `decorator_func`.
- `decorator_func(12, 15)(my_fun)('Geeks', 'for', 'Geeks')12`: 이 줄은 매개변수와 를 사용하여 데코레이터를 적용합니다 `15`. 데코레이터는 인수 , , 를 사용하여 `my_fun`호출하기 전에 메시지와 합계를 출력합니다 .`my_fun'Geeks''for''Geeks'`

```python
# Python code to illustrate
# Decorators with parameters in Python

def decorator_func(x, y):

    def Inner(func):

        def wrapper(*args, **kwargs):
            print("I like Geeksforgeeks")
            print("Summation of values - {}".format(x+y) )

            func(*args, **kwargs)

        return wrapper
    return Inner


# Not using decorator
def my_fun(*args):
    for ele in args:
        print(ele)

# another way of using decorators
decorator_func(12, 15)(my_fun)('Geeks', 'for', 'Geeks')
```

산출

예시 3:

코드 설명은 다음과 같습니다.

1. **데코레이터 함수** :type_check_decorator
   - **매개변수** : expected_type모든 인수의 유형을 지정합니다.
   - **내부 기능** : 장식될 기능을 가져옵니다 .decoratorfunc
   - **래퍼 함수** : 모든 인수가 맞는지 확인하고 , 맞는 경우 호출합니다. 그렇지 않으면 반환합니다 .wrapperexpected_typefunc"Invalid Input"

2. **장식된 함수** :
   - string_join: 모든 문자열 인수를 하나의 문자열로 결합합니다.
   - summation: 모든 정수 인수를 합산합니다.

3. **함수 호출** :
   - string_join("I ", 'like ', "Geeks", 'for', "geeks"): 문자열을 올바르게 연결합니다.
   - summation(19, 2, 8, 533, 67, 981, 119): 정수를 정확하게 합산합니다.

```python
# Simple decorator with parameters
def type_check_decorator(expected_type):
    def decorator(func):
        def wrapper(*args, **kwargs):
            if all(isinstance(arg, expected_type) for arg in args):
                return func(*args, **kwargs)
            return "Invalid Input"
        return wrapper
    return decorator

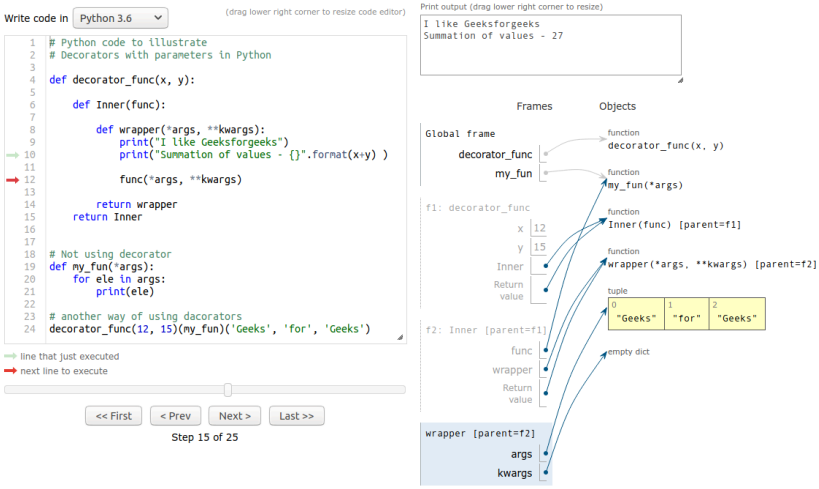

@type_check_decorator(str)
def string_join(*args):
    return ''.join(args)


@type_check_decorator(int)
def summation(*args):
    return sum(args)

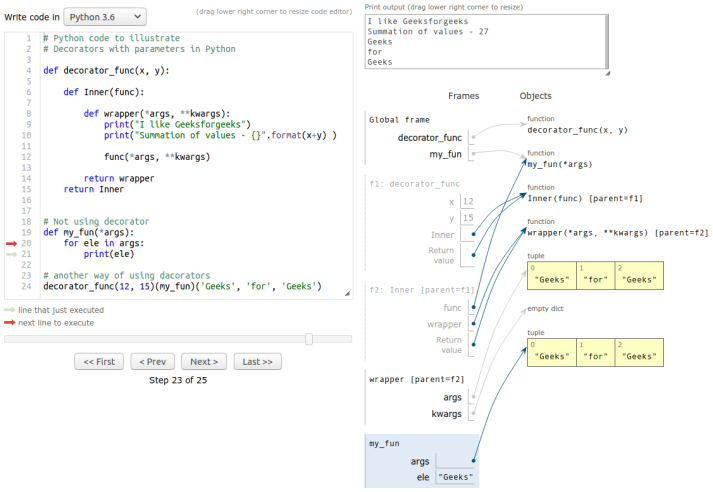

# Test the functions
print(string_join("I ", 'like ', "Geeks", 'for', "geeks"))
print(summation(19, 2, 8, 533, 67, 981, 119))
```

산출

1. 데코레이터 내부



2. 함수 내부



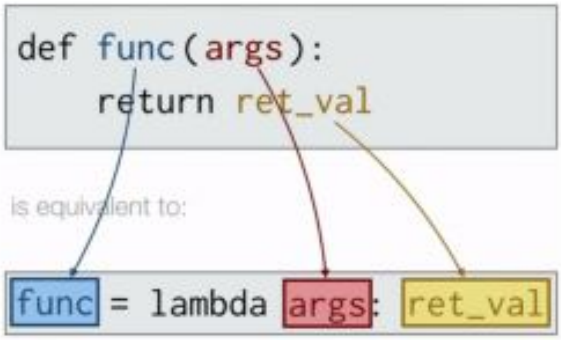
파이썬 람다 함수

파이썬 람다 함수 는 익명 함수입니다. 즉, 함수에 이름이 없습니다. 이미 알고 있듯이 *def* 키워드는 파이썬에서 일반 함수를 정의하는 데 사용됩니다. 마찬가지로, *lambda 키워드는* 파이썬에서 익명 함수를 정의하는 데 사용됩니다 .

파이썬 람다 함수 구문

```
lambda argument : expression
```

- **람다:** 함수를 정의하는 키워드.
- **인수:** 입력 매개변수의 쉼표로 구분된 목록(일반 함수와 같음).
- **표현식:** 평가되어 반환되는 단일 표현식입니다.



예 :

```
# Using lambda
sq = lambda x: x ** 2
print(sq(3))

# Using def
def sqdef(x):
    return x ** 2
print(sqdef(3))
```

예

a 인수와 인수를 곱하고 b 결과를 반환합니다.

```
x = lambda a, b : a * b
print(x(5, 6))
```

표준 함수에서는 함수 이름 뒤에 괄호를 붙여 함수를 호출합니다. 하지만 lambda 함수에는 이름이 없습니다! 대신 표현식을 괄호로 묶고 표준 함수 이름을 다루는 것처럼 사용할 수 있습니다.

```
def do_boring_stuff(x):
    return 42 * x

print(
    do_boring_stuff(10)
)
# OUTPUT:
# 420

print(
    (lambda x: 42 * x)(10)
)
# OUTPUT:
# 420
```

함수에 이름을 붙여야 한다면 표준 함수를 사용하는 것이 좋습니다

```
# Demo only. Please don't do this!
i_want_to_name_this = lambda x: 42 * x

print(
    i_want_to_name_this(10)
)
# OUTPUT:
# 420
```

람다의 강력함은 다른 함수 내부에서 익명 함수로 사용할 때 더 잘 드러납니다.

하나의 인수를 받는 함수 정의가 있고, 해당 인수가 알 수 없는 숫자와 곱해진다고 가정해 보겠습니다.

```
def myfunc(n):
  return lambda a : a * n
```

해당 함수 정의를 사용하여 항상 입력한 숫자를 두 배로 늘리는 함수를 만듭니다.

예

```
def myfunc(n):
  return lambda a : a * n

mydoubler = myfunc(2)

print(mydoubler(11))
```

또는 동일한 함수 정의를 사용하여 항상 입력한 숫자를 *세 배로 늘리는* 함수를 만듭니다 .

예

```
def myfunc(n):
  return lambda a : a * n

mytripler = myfunc(3)

print(mytripler(11))
```

람다와 def 키워드의 차이점

lambda는 간결하지만 복잡한 로직을 처리할 때는 def보다 성능이 떨어집니다. 두 가지를 간략하게 비교해 보겠습니다.

Feature	`lambda` Function	Regular Function (`def`)
Definition	Single expression with `lambda`.	Multiple lines of code.
Name	Anonymous (or named if assigned).	Must have a name.
Statements	Single expression only.	Can include multiple statements.
Documentation	Cannot have a docstring.	Can include docstrings.
Reusability	Best for short, temporary functions.	Better for reusable and complex logic.

리스트 이해력을 갖춘 람다

람다를 리스트 컴프리헨션 과 결합하면 간결한 방식으로 데이터에 변환을 적용할 수 있습니다.

예:

```
li = [lambda arg=x: arg * 10 for x in range(1, 5)]
for i in li:
    print(i())
```

산출

```
10
20
30
40
```

설명:

- 람다 함수는 각 요소를 제곱합니다.
- 리스트 이해는 li를 반복하고 각 요소에 람다를 적용합니다.
- 이는 데이터 전처리 또는 조작 작업에서 데이터 세트에 변환을 적용하는 데 이상적입니다.

if-else를 사용한 람다

람다 함수는 조건 논리를 직접 통합할 수 있으므로 함수 내에서 간단한 의사 결정을 처리할 수 있습니다.

예:

```
# Example: Check if a number is even or odd
check = lambda x: "Even" if x % 2 == 0 else "Odd"

print(check(4))
print(check(7))
```

산출

설명:

- 람다는 숫자가 2로 나누어 떨어지는지 확인합니다(x % 2 == 0).
- 참이면 "짝수"를 반환하고, 그렇지 않으면 "홀수"를 반환합니다.
- 이 접근 방식은 간단한 조건에 따라 값을 레이블로 지정하거나 분류하는 데 유용합니다.

여러 문장이 있는 람다

예:

```
# Example: Perform addition and multiplication in a single line
calc = lambda x, y: (x + y, x * y)

res = calc(3, 4)
print(res)
```

산출

```
(7, 12)
```

설명:

- 람다 함수는 덧셈과 곱셈을 모두 수행하고 두 결과를 모두 포함하는 튜플을 반환합니다.
- 이 기능은 여러 계산을 수행하여 함께 반환해야 하는 시나리오에 유용합니다.

람다 함수는 filter() , map() , reduce() 와 같은 내장 함수와 함께 사용할 수 있습니다 .

filter()와 함께 람다 사용

Python의 filter () 함수는 함수와 리스트를 인수로 받습니다. 이 함수는 "sequence" 시퀀스의 모든 요소를 필터링하는 우아한 방법을 제공하며, 해당 함수가 True를 반환합니다.

예:

```
# Example: Filter even numbers from a list
n = [1, 2, 3, 4, 5, 6]
even = filter(lambda x: x % 2 == 0, n)
print(list(even))
```

산출

```
[2, 4, 6]
```

설명:

- 람다 함수는 숫자가 짝수인지 확인합니다(x % 2 == 0).
- filter()는 이 조건을 nums의 각 요소에 적용합니다.

map()과 함께 람다 사용하기

Python의 map () 함수 는 함수와 리스트를 인수로 받습니다. 이 함수는 람다 함수와 함께 호출되며, 각 항목에 대해 해당 함수가 반환한 람다 함수로 수정된 모든 항목을 포함하는 새 리스트가 반환됩니다.

예:

```

```

```python
# Example: Double each number in a list
a = [1, 2, 3, 4]
b = map(lambda x: x * 2, a)
print(list(b))
```

산출

```
[2, 4, 6, 8]
```

설명:

- 람다 함수는 각 숫자를 두 배로 늘립니다.
- map()은 a를 반복하고 변환을 적용합니다.

파이썬의 반복자

파이썬에서 반복자는 컬렉션(리스트, 튜플, 딕셔너리 등)의 모든 요소를 한 번에 하나씩 탐색하는 데 사용되는 객체입니다. 반복자는 두 가지 주요 메서드를 포함하는 반복자 프로토콜을 따릅니다.

- **__iter__():** 반복자 객체 자체를 반환합니다.
- **__next__():** 시퀀스의 다음 값을 반환합니다. 시퀀스가 끝나면 StopIteration을 발생시킵니다.

파이썬에서 반복자가 필요한 이유는 무엇입니까?

- **지연 평가:** 필요할 때만 항목을 처리하여 메모리를 절약합니다.
- **생성기 통합:** 생성기 및 기능적 도구와 잘 어울립니다.
- **상태 추적:** 호출 간 위치를 기억합니다.
- **균일 루핑:** 동일한 구문을 사용하여 여러 데이터 유형에서 작동합니다.
- **구성 가능한 논리:** itertools와 같은 도구를 사용하여 복잡한 파이프라인을 쉽게 구축합니다.

내장 반복자 예제

```
s = "GFG"
it = iter(s)

print(next(it))
print(next(it))
print(next(it))
```

산출

사용자 정의 반복자 만들기

Python에서 사용자 정의 반복자를 생성하려면 Python 반복자 프로토콜에 따라 __iter__() 및 __next__() 메서드를 구현하는 클래스를 정의해야 합니다.

따라야 할 단계:

- **클래스 정의:** 반복자 역할을 할 클래스를 정의하는 것으로 시작합니다.
- **속성 초기화:** 클래스의 __init__() 메서드에서 반복 프로세스 전반에 사용될 필수 속성을 초기화합니다.
- **__iter__() 구현:** 이 메서드는 반복자 객체 자체를 반환해야 합니다. 일반적으로 self를 반환하는 것만큼 간단합니다.
- **__next__() 구현:** 이 메서드는 호출될 때마다 시퀀스의 다음 항목을 제공해야 합니다.

아래는 2부터 시작하여 짝수를 반복하는 EvenNumbers라는 사용자 정의 클래스의 예입니다.

```
class EvenNumbers:
    def __iter__(self):
        self.n = 2  # Start from the first even number
        return self

    def __next__(self):
        x = self.n
        self.n += 2  # Increment by 2 to get the next even number
        return x

# Create an instance of EvenNumbers
even = EvenNumbers()
it = iter(even)

# Print the first five even numbers
print(next(it))
print(next(it))
print(next(it))
print(next(it))
print(next(it))
```

산출

```
2
4
6
8
10
```

설명:

- **초기화:** __iter__() 메서드는 반복자를 첫 번째 짝수인 2로 초기화합니다.
- **반복:** __next__() 메서드는 현재 숫자를 검색한 다음 2만큼 증가시켜 다음 호출에서 후속 짝수가 반환되도록 합니다.
- **사용법:** EvenNumbers의 인스턴스를 생성하고 이를 반복자로 바꾼 다음 next() 함수를 사용하여 한 번에 하나씩 짝수를 가져옵니다.

StopIteration 예외

StopIteration 예외는 Python의 반복자 프로토콜과 통합되어 있습니다. 이 예외는 반복자가 더 이상 반환할 항목이 없음을 나타냅니다. 이 예외가 발생하면 동일한 반복자에 대해 next()를 추가로 호출해도 StopIteration 예외가 계속 발생합니다.

예:

```
li = [100, 200, 300]
it = iter(li)

# Iterate until StopIteration is raised
while True:
    try:
        print(next(it))
    except StopIteration:
        print("End of iteration")
        break
```

산출

```
100
200
300
반복의 끝
```

Iterator와 Iterable의 차이점

- 반복 가능한 객체는 반복될 수 있는 객체입니다(예: for 루프).
- 반복자는 반복 가능한 객체에 대해 iter()를 호출했을 때 반환되는 객체입니다.

특징	반복 가능	반복자
정의	반복자를 반환할 수 있는 모든 객체	반복을 위한 상태가 있는 객체
주요 방법	__iter__()를 구현합니다	__iter__()와 __next__()를 모두 구현합니다.
예시	리스트, 튜플, 문자열, 사전, 세트	iter()에서 반환된 객체